JobSheet - Week 3 - Inheritance , Abstract Class , Interface

Nama: <Tuliskan Nama Lengkap di sini>

NIM: <Tuliskan NIM>

Kelas: 1A/1B/1C/1D

Repo GitHub: <Tuliskan Tautan Repo GitHub>

Instruksi Pengerjaan:

1. Kerjakan 3 soal di bawah ini dengan melengkapi setiap kolom jawaban yang disediakan pada jobsheet ini.
2. Jawaban setiap soal mencakup source code, screenshot hasil dari program yang ditampilkan full screen termasuk taskbar (tambahkan beberapa screenshot jika diperlukan), penjelasan permasalahan dan solusi yang dihadapi, nama teman yang membantu memecahkan masalah (opsional).
3. Dikumpulkan pada Assignment Classroom sesuai dengan deadline yang tertera pada assignment tersebut.
4. Format penamaan file jobsheet: W3_P_<Kelas 1X>_<3 Digit_NIM_Terakhir>.docx/pdf. Contoh: W3_P_1B_001.docx/pdf.
5. Submit semua jawaban dalam bentuk file java pada repository GitHub masing-masing.

No. 1 Inheritance

Soal Praktikum

Dalam latihan ini, sebuah *subclass* bernama **Cylinder** diturunkan dari *superclass* **Circle** seperti yang ditunjukkan pada diagram kelas (di mana terdapat tanda panah yang menunjuk ke atas dari *subclass* ke *superclass*-nya).
 	
Pelajari bagaimana *subclass* Cylinder memanggil *constructor* dari *superclass* (melalui super() dan super(radius)) serta mewarisi variabel dan *method* dari *superclass* Circle. Berikut adalah kode sumber untuk **Circle.java, Cylinder.java** dan **TestCylinder.java**:
 	
1. Circle.java
/**
 * The Circle class models a circle with a radius and color.
 */
public class Circle { // Save as "Circle.java"
  // private instance variable, not accessible from outside this class
  private double radius;

```
    private String color;

    // Constructors (overloaded)
    /** Constructs a Circle instance with default value for radius and color */
    public Circle() { // 1st (default) constructor
        radius = 1.0;
        color = "red";
    }

    /** Constructs a Circle instance with the given radius and default color */
    public Circle(double r) { // 2nd constructor
        radius = r;
        color = "red";
    }

    /** Returns the radius */
    public double getRadius() {
        return radius;
    }

    /** Returns the area of this Circle instance */
    public double getArea() {
        return radius * radius * Math.PI;
    }

    /** * Return a self-descriptive string of this instance in the form of
     * Circle[radius=?,color=?]
     */
    public String toString() {
        return "Circle[radius=" + radius + " color=" + color + "]";
    }
}
```

2. Cylinder.java

```
public class Cylinder extends Circle { // Save as "Cylinder.java"
    private double height; // private variable

    // Constructor with default color, radius and height
    public Cylinder() {
        super(); // call superclass no-arg constructor Circle()
        this.height = 1.0;
    }

    // Constructor with default radius, color but given height
    public Cylinder(double height) {
        super(); // call superclass no-arg constructor Circle()
        this.height = height;
    }

    // Constructor with default color, but given radius, height
    public Cylinder(double radius, double height) {
        super(radius); // call superclass constructor Circle(radius)
        this.height = height;
    }

    // A public method for retrieving the height
```

```
    public double getHeight() {
        return height;
    }

    // A public method for computing the volume of cylinder
    // use superclass method getArea() to get the base area
    public double getVolume() {
        return getArea() * height;
    }
}
```

3. TestCylinder.java

```
public class TestCylinder { // save as "TestCylinder.java"
    public static void main(String[] args) {

        // Declare and allocate a new instance of cylinder
        // with default color, radius, and height
        Cylinder c1 = new Cylinder();
        System.out.println("Cylinder:"
            + " radius=" + c1.getRadius()
            + " height=" + c1.getHeight()
            + " base area=" + c1.getArea()
            + " volume=" + c1.getVolume());

        // Declare and allocate a new instance of cylinder
        // specifying height, with default color and radius
        Cylinder c2 = new Cylinder(10.0);
        System.out.println("Cylinder:"
            + " radius=" + c2.getRadius()
            + " height=" + c2.getHeight()
            + " base area=" + c2.getArea()
            + " volume=" + c2.getVolume());

        // Declare and allocate a new instance of cylinder
        // specifying radius and height, with default color
        Cylinder c3 = new Cylinder(2.0, 10.0);
        System.out.println("Cylinder:"
            + " radius=" + c3.getRadius()
            + " height=" + c3.getHeight()
            + " base area=" + c3.getArea()
            + " volume=" + c3.getVolume());
    }
}
```

A. Instruksi - Modifikasi Class Circle:

1. Modifikasi class Circle, tambahkan:
    - variable color : string
    - Constructor Circle(radius : double, color : string)
    - Getter and setter untuk color
2. Implementasikan class Cylinder! Jelaskan keterhubungannya dengan class Circle!
3. Konsep apa yang diterapkan pada constructor Cylinder? Jelaskan cara kerja, kelebihan dan kekurangannya!

4. Tuliskan sebuah program pengujian (sebut saja TestCylinder) untuk menguji kelas Cylinder yang telah dibuat!

B. Instruksi – Overriding method getArea():

Metode Overriding dan "Super": Subclass **Cylinder** mewarisi metode getArea() dari superclass **Circle**.

- Cobalah melakukan *overriding* metode getArea() pada subclass Cylinder untuk menghitung luas permukaan (2π × radius × tinggi + 2 × luas alas) tabung, alih-alih luas alasnya saja!

Artinya, jika getArea() dipanggil oleh instans **Circle**, maka ia akan mengembalikan luas lingkaran. Namun, jika getArea() dipanggil oleh instans **Cylinder**, ia akan mengembalikan luas permukaan tabung.

Jika Anda melakukan *override* pada getArea() di subclass Cylinder, maka metode getVolume() tidak akan lagi berfungsi dengan benar. Hal ini terjadi karena getVolume() akan menggunakan metode getArea() yang sudah ditimpa (*overridden*) yang ditemukan di kelas yang sama. (Runtime Java hanya akan mencari ke superclass jika tidak dapat menemukan metode tersebut di kelas ini).

- Perbaikilah metode getVolume() tersebut!

Petunjuk: Setelah melakukan overriding pada metode getArea() di subclass Cylinder, Anda dapat memilih untuk memanggil metode getArea() dari superclass Circle dengan cara memanggil super.getArea().

C. Instruksi – Overriding method getArea():

Sediakan metode toString() pada kelas Cylinder, yang menimpa (override) metode toString() yang diwarisi dari superclass Circle, sebagai contoh:

```
@Override
public String toString() { // di dalam kelas Cylinder
   return "Cylinder: subclass of " + super.toString() // menggunakan toString() milik Circle
         + " height=" + height;
}
```

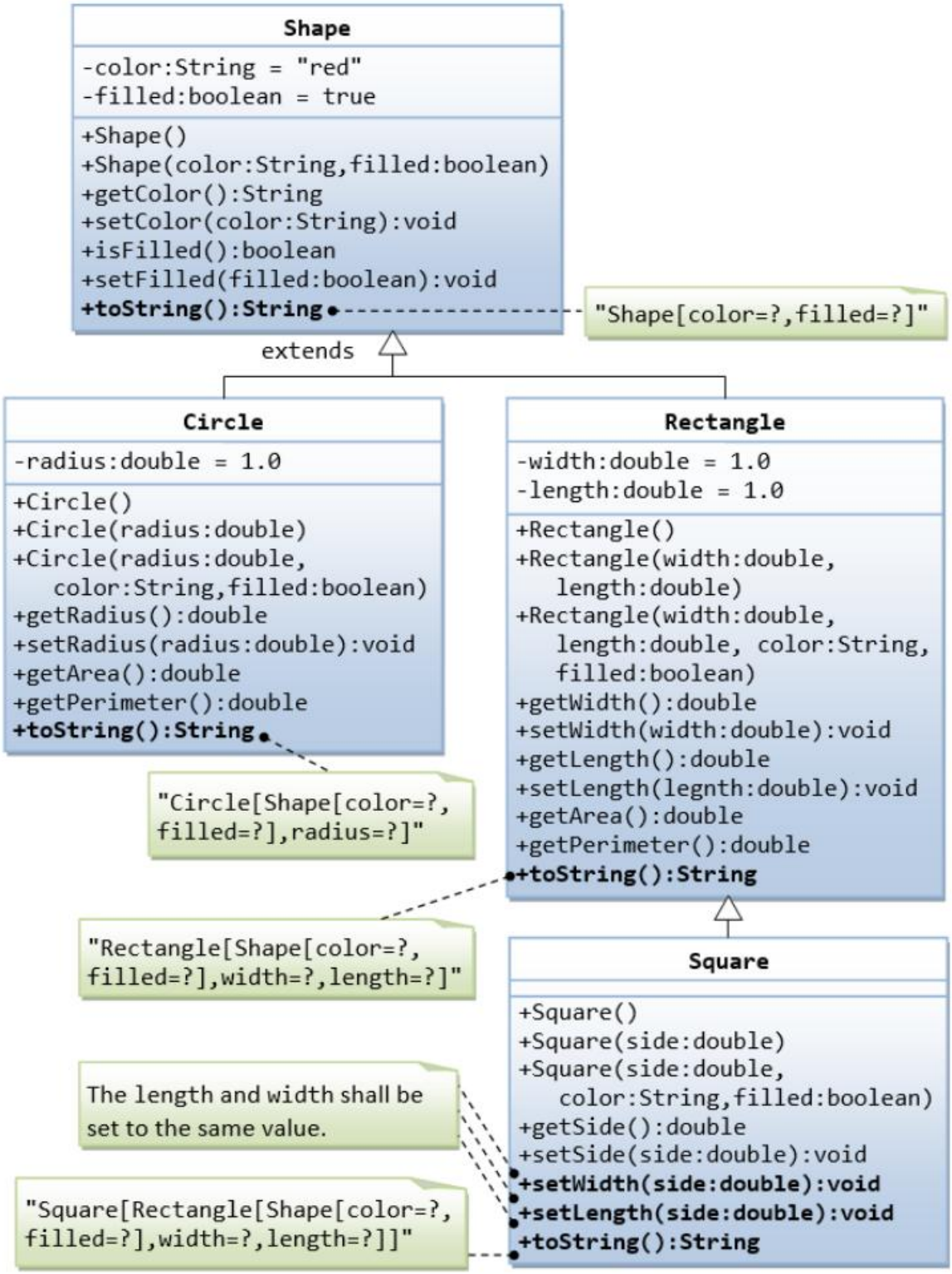
- Cobalah metode toString() tersebut di dalam kelas TestCylinder!

Source Code
Screenshot Hasil
Penjelasan Permasalahan dan Solusi
Nama Teman Hal yang Dibantu (Opsional)

No. 2 Superclass dan Subclass

Soal Praktikum

Superclass Shape dan Subclass-nya: Circle, Rectangle, dan Square.



A. Instruksi - Hirarki Kelas Shape:

1. Buatlah sebuah *superclass* bernama **Shape** (seperti yang ditunjukkan pada diagram kelas), yang berisi:

   - Dua variabel instans: color (String) dan filled (boolean).
   - Dua *constructor*: sebuah *no-arg (no-argument) constructor* yang menginisialisasi color ke "green" dan filled ke true, dan sebuah *constructor* yang menginisialisasi color dan filled ke nilai yang diberikan.
   - *Getter* dan *setter* untuk semua variabel instans. Berdasarkan konvensi, *getter* untuk variabel boolean xxx disebut isXXX() (bukan getXxx() seperti pada tipe lainnya).
   - Sebuah metode toString() yang mengembalikan "A Shape with color of xxx and filled/Not filled".
   - Tulislah sebuah program uji untuk menguji semua metode yang didefinisikan dalam Shape.
2. Buatlah dua *subclass* dari Shape bernama **Circle** dan **Rectangle**, seperti yang ditunjukkan pada diagram kelas.
   Kelas **Circle** berisi:
   - Sebuah variabel instans radius (double).
   - Tiga *constructor* seperti yang ditunjukkan. *No-arg constructor* menginisialisasi radius ke 1.0.
   - *Getter* dan *setter* untuk variabel instans radius.
   - Metode getArea() dan getPerimeter().
   - *Override* metode toString() yang diwarisi, untuk mengembalikan "A Circle with radius=xxx, which is a subclass of yyy", di mana yyy adalah *output* dari metode toString() milik *superclass*.

   Kelas **Rectangle** berisi:
   - Dua variabel instans width (double) dan length (double).
   - Tiga *constructor* seperti yang ditunjukkan. *No-arg constructor* menginisialisasi width dan length ke 1.0.
   - *Getter* dan *setter* untuk semua variabel instans.
   - Metode getArea() dan getPerimeter().
   - *Override* metode toString() yang diwarisi, untuk mengembalikan "A Rectangle with width=xxx and length=zzz, which is a subclass of yyy", di mana yyy adalah *output* dari metode toString() milik *superclass*.
3. Buatlah sebuah kelas bernama **Square**, sebagai *subclass* dari Rectangle. Yakinkan diri Anda bahwa Square dapat dimodelkan sebagai *subclass* dari Rectangle. Square tidak memiliki variabel instans, tetapi mewarisi variabel instans width dan length dari *superclass*-nya, Rectangle.
   - Sediakan *constructor* yang sesuai (seperti yang ditunjukkan dalam diagram kelas)! Petunjuk:

     ```
     public Square(double side) {
       super(side, side);  // Call superclass Rectangle(double, double)
     ```

}

- *Override* metode toString() untuk mengembalikan "A Square with side=xxx, which is a subclass of yyy", di mana yyy adalah *output* dari metode toString() milik *superclass*!
- Apakah Anda perlu melakukan *override* pada getArea() dan getPerimeter()? Cobalah!
- *Override* setLength() dan setWidth() untuk mengubah width sekaligus length, guna menjaga geometri persegi!

Source Code
Screenshot Hasil
Penjelasan Permasalahan dan Solusi
Nama Teman Hal yang Dibantu (Opsional)

No. 3 Multiple Inheritance

Soal Praktikum

Dua contoh di bawah merupakan kelas pertama yang mendeskripsikan sekumpulan Karyawan (Employees) yang bekerja di sebuah pabrik, dan kelas kedua yang mendeskripsikan subset Karyawan yang merupakan Manajer (Manager) di pabrik yang sama. Kedua kelas tersebut mewakili sebuah contoh untuk menunjukkan kekuatan ekspresif dari Pewarisan (Inheritance) dalam Java.
 	
Secara khusus, kelas Manager didefinisikan sebagai subclass dari Employee, dengan tujuan untuk menggunakan kembali perangkat lunak yang telah ditulis untuk kelas Employee.
 	
1. Class Employee.java
class Employee {
  private String name;
  private double salary;
  private int hireday;
  private int hiremonth;
  private int hireyear;
 	
  public Employee(String n, double s, int day, int month, int year) {
  name = n;
  salary = s;
  hireday = day;
  hiremonth = month;
  hireyear = year;
  }
 	
  public void print() {
  System.out.println(name + " " + salary + " " + hireYear());
  }
 	
  public void raiseSalary(double byPercent) {
  salary *= 1 + byPercent / 100;
  }
 	
  public int hireYear() {
  return hireyear;
  }
}
 	
2. Class EmployeeTest.java
public class EmployeeTest {
  public static void main(String[] args) {
  // Mendeklarasikan dan mengalokasikan array untuk 3 objek Employee
  Employee[] staff = new Employee[3];
 	
  // Inisialisasi data karyawan
  staff[0] = new Employee("Antonio Rossi", 2000000, 1, 10, 1989);
  staff[1] = new Employee("Maria Bianchi", 2500000, 1, 12, 1991);
  staff[2] = new Employee("Isabel Vidal", 3000000, 1, 11, 1993);

```
        // Menaikkan gaji setiap staf sebesar 5%
        for (int i = 0; i < 3; i++) {
            staff[i].raiseSalary(5);
        }

        // Mencetak data dari setiap staf
        for (int i = 0; i < 3; i++) {
            staff[i].print();
        }
    }
}
```

3. Class Manager.java

```
import java.util.Calendar;
import java.util.GregorianCalendar;

class Manager extends Employee {
    private String secretaryName;

    public Manager(String n, double s, int d, int m, int y) {
        super(n, s, d, m, y);
        secretaryName = "";
    }

    @Override
    public void raiseSalary(double byPercent) {
        // Menambahkan bonus 1/2% untuk setiap tahun masa kerja
        GregorianCalendar todaysDate = new GregorianCalendar();
        int currentYear = todaysDate.get(Calendar.YEAR);
        double bonus = 0.5 * (currentYear - hireYear());

        super.raiseSalary(byPercent + bonus);
    }

    public String getSecretaryName() {
        return secretaryName;
    }
}
```

4. Class ManagerTest.java

```
public class ManagerTest {
    public static void main(String[] args) {
        // Mendeklarasikan dan mengalokasikan array untuk 3 objek Employee
        Employee[] staff = new Employee[3];

        // Mengisi array dengan kombinasi Employee dan Manager (Polimorfisme)
        staff[0] = new Employee("Antonio Rossi", 2000000, 1, 10, 1989);
        staff[1] = new Manager("Maria Bianchi", 2500000, 1, 12, 1991);
        staff[2] = new Employee("Isabel Vidal", 3000000, 1, 11, 1993);

        // Menaikkan gaji semua staf sebesar 5%
        for (int i = 0; i < 3; i++) {
            staff[i].raiseSalary(5);
        }

        // Mencetak data setiap staf
```

```
        for (int i = 0; i < 3; i++) {
            staff[i].print();
        }
    }
}
```

A. Instruksi 1

Terdapat sebuah abstract class bernama Sortable.

```
abstract class Sortable {
    public abstract int compare(Sortable b);

    public static void shell_sort(Sortable[] a) {
        // Shell sort body
    }
}
```

Ketika Sortable diturunkan ke kelas Employee, metode compare akan diimplementasikan.

```
class Employee extends Sortable {
    /* another methods */

    @Override
    public int compare(Sortable b) {
        Employee eb = (Employee) b;
        if (salary < eb.salary) return -1;
        if (salary > eb.salary) return 1;
        return 0;
    }
}
```

Silakan coba kode di atas! Panggil metode compare, di dalam kelas EmployeeTest!

B. Instruksi 2

Bayangkan kita ingin mengurutkan Manager dengan cara yang serupa:

```
class Managers extends Employee extends Sortable
```

1. Apakah itu akan berhasil?
2. Apa solusi Anda?

Source Code
Screenshot Hasil
Penjelasan Permasalahan dan Solusi
Nama Teman Hal yang Dibantu (Opsional)